

◆ EXCEPTION HANDLER METHOD

```
@ExceptionHandler(Exception.class)
public ResponseEntity<EmailResponseDTO> handleException(Exception e) {
```

This is the most important method in the class.

This method tells Spring:

```
If an Exception occurs,
send it here.
```

◆ WHAT IS @ExceptionHandler ?

```
@ExceptionHandler(Exception.class)
```



This annotation tells Spring:

```
I can handle Exception objects.
```

Think of it as a rule.

Rule:

```
If Exception happens
↓
Call handleException()
```

◆ WHAT DOES Exception.class MEAN?

```
Exception.class
```

represents the Exception type.

Remember:

```
Exception
```

is the parent class of many errors.

Examples:

```
NullPointerException
```

```
RuntimeException
```

```
IllegalArgumentException
```

```
IOException
```

All of these are related to Exception.

By writing:

```
@ExceptionHandler(Exception.class)
```

we are saying:

```
Catch any exception.
```

It acts like a safety net.

◆ WHAT DOES SPRING DO?

Suppose somewhere in your project:

```
throw new RuntimeException();
```

occurs.

Spring sees:

```
An Exception occurred.
```

Then **Spring searches:**

```
Do I have an ExceptionHandler?
```

It finds:

```
@ExceptionHandler(Exception.class)
```

and immediately **calls:**

```
handleException(...)
```

◆ METHOD NAME

```
handleException
```

This is simply the method name.

It could have been:

```
errorHandler
```

or

```
myExceptionHandler
```

Spring does not care about the name.

The annotation is what matters.

◆ WHAT IS Exception e ?

Exception e

This parameter stores the exception that occurred.

Suppose Flask is unavailable.

An exception occurs:

Connection refused

Spring passes that exception into:

Exception e

Think:

Exception
↓
Stored inside e

◆ WHAT CAN e CONTAIN?

Example:

NullPointerException

Then:

e

contains information about that error.

Example:

Connection timeout

Then:

e

contains information about that error.

Example:

Flask server unavailable

Then:

e

contains information about that error.

◆ WHY PASS e IF WE DON'T USE IT?

Good observation.

Your code:

```
public ResponseEntity<EmailResponseDTO>
handleException(Exception e)
```



receives:

```
e
```

but never uses it.

This is completely valid.

The developer included it because:

Spring automatically provides the exception.

Future version:

```
System.out.println(e.getMessage());
```

or

```
logger.error(e.getMessage());
```

could be added.

For now:

```
e
```

is simply ignored.

◆ RETURN TYPE

```
ResponseEntity<EmailResponseDTO>
```

Look carefully.

This method promises:

```
I will return a ResponseEntity
containing an EmailResponseDTO.
```

Exactly like your controller methods.

Therefore somewhere inside this method we must return:

```
ResponseEntity<EmailResponseDTO>
```

object.

◆ WHEN IS THIS METHOD CALLED?

Normal Request:

User
↓
Controller
↓
Service
↓
Success
↓
Response

Exception Request:

User
↓
Controller
↓
Service
↓
Exception
↓
Spring
↓
handleException(Exception e)
↓
Error Response

◆ REAL PROJECT EXAMPLE

Suppose:

```
restTemplate.postForEntity(...)
```

fails because Flask is not running.

Exception occurs.

Spring detects:

```
Exception
```

Spring finds:

```
@ExceptionHandler(Exception.class)
```

Spring calls:

```
handleException(e)
```

Now this method creates the response instead of letting the application crash.

◆ SUMMARY

```
@ExceptionHandler(Exception.class)
```

tells Spring to send exceptions to this method.

```
Exception.class
```

means any Exception can be handled.

```
Exception e
```

contains the actual error object.

```
ResponseEntity<EmailResponseDTO>
```

means the method returns an HTTP response containing an EmailResponseDTO.

When an exception occurs, Spring automatically calls this method and lets it build a custom error response.

The next section should continue with:

```
return ResponseEntity
    .status(HttpStatus.INTERNAL_SERVER_ERROR)
    .body(new EmailResponseDTO("SERVER_ERROR", 0.0));
```

because this is where the actual error response is created and returned to the client.